



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“CONECCIÓN DE SQL SERVER CON VISUAL
STUDIO C#”

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I.	INFORMACIÓN GENERAL	3
-	Objetivos	3
-	Equipos, materiales, programas y recursos utilizados	3
II.	MARCO TEORICO	3
III.	PROCEDIMIENTO	4
	Programa: Negocio:.....	4
	1. Diseño del Programa.....	4
	2. Tabla de Controles.....	5
	3. Código del Programa.....	5
	Programa: Negocio_version2:.....	8
	1. Diseño del Programa.....	8
	2. Tabla de Controles.....	8
	3. Código del Programa.....	9
IV.	ANALISIS E INTERPRETACION DE RESULTADOS	16
	CONCLUSIONES	16
	RECOMENDACIONES	16
	WEBGRAFIA	17



INFORME DE LABORATORIO

TEMA: CONECCIÓN DE SQL SERVER CON VISUAL STUDIO C#

I. INFORMACIÓN GENERAL

- Objetivos:

- Desarrollar un sistema de control de productos asociado a SQL Server.
- Implementar las operaciones CRUD (Crear, Leer, Actualizar y Borrar).
- Establecer conexión entre Windows Forms y bases de datos.
- Usar comandos SQL desde C#.
- Implementar validaciones de datos en formularios.
- Mostrar información almacenada a través de DataGridView.
- Reforzar conocimientos acerca de la manipulación de bases de datos en aplicaciones de escritorio.

- Equipos, materiales, programas y recursos utilizados:

- Computadora con sistema operativo Windows 8
- Visual Studio
- Lenguaje de programación C#.
- .NET Framework

II. MARCO TEORICO

Base de Datos: Una base de datos es un sistema organizado en el cual se almacena información de manera estructurada.

SQL Server: Es un sistema gestor de bases de datos de Microsoft.

Permite:

- Guardar información
- Modificar registros.
- Consultar datos.
- Borrar información.

CRUD: Las aplicaciones CRUD realizan las siguientes cuatro operaciones principales:

- Create: Insertar registros.
- Read: Consultar registros.
- Update: Modificar registros.
- Delete: Borrar registros.



Windows Forms: Windows Forms se utiliza para crear aplicaciones de escritorio con interfaz gráfica desde C#.

ADO.NET: ADO.NET se utiliza para establecer conexiones entre aplicaciones .NET y bases de datos.

Clases utilizadas:

- SqlConnection
- SqlCommand
- SqlDataAdapter
- DataTable

DataGridView: Control visual que permite mostrar tablas dentro de la aplicación.

III. PROCEDIMIENTO

Programa: Negocio:

1. Diseño del Programa

Mantenimiento de Productos

Codigo

Nombre

Marca

Precio



2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Mantenimiento de Productos
Label2	Text	Codigo
Label3	Text	Nombre
Label4	Text	Marca
Label5	Text	Precio
Button1	Name	btnAgregar
	Text	Agregar
Button2	Name	btnEliminar
	Text	Eliminar
Button3	Name	btnModificar
	Text	Modificar

3. Código del Programa

Funcionalidad 1: Importación de librería SQL

Permite utilizar las clases necesarias para trabajar con SQL Server.

```
using Microsoft.Data.SqlClient;
```

Funcionalidad 2: Constructor principal

Inicializa todos los componentes visuales del formulario.

```
public Form1()
{
    InitializeComponent();
}
```

Funcionalidad 3: Conexión a la base de datos

Esta línea crea la conexión con SQL Server.

```
SqlConnection conexion = new SqlConnection(
    "Server=localhost;Database=negocio;Integrated Security=true;Encrypt=false");
```

Elementos de la conexión

- Server=localhost: Servidor SQL local.
- Database=negocio: Base de datos utilizada.
- Integrated Security=true: Usa autenticación de Windows.
- Encrypt=false: Desactiva cifrado de conexión.

Funcionalidad 4: Evento btnAgregar_Click

Abrir conexión

```
1 referencia
private void btnAgregar_Click(object sender, EventArgs e)
{
    conexion.Open();
}
```

Consulta SQL INSERT

Inserta un nuevo producto en la tabla productos.

```
string consulta = "insert into productos values ('" + textBox1.Text + "','" + textBox2.Text + "','" + textBox3.Text + "','" + textBox4.Text + "')";
```

Creación del comando SQL

Asocia la consulta SQL con la conexión.

```
SqlCommand comando = new SqlCommand(consulta, conexion);
```

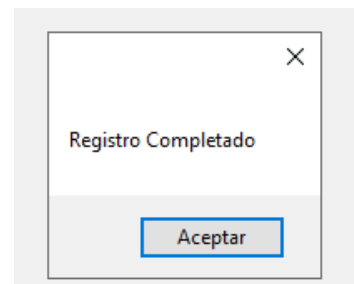
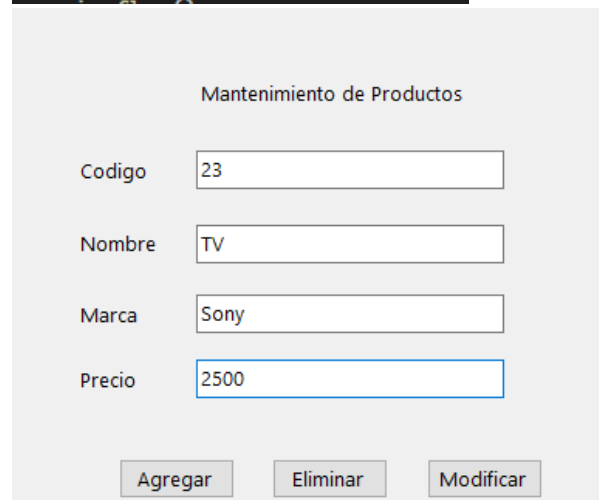
Ejecución de la consulta

Ejecuta consultas que no retornan datos.

```
comando.ExecuteNonQuery();
```

Mensaje de confirmación

```
MessageBox.Show("Registro Completado");
```



Cerrar conexión

El sistema agrega correctamente nuevos productos en la base de datos.

```
conexion.Close();  
}
```

Funcionalidad 5: Evento btnEliminar_Click

Consulta DELETE

Elimina registros según el código ingresado.

```
private void btnEliminar_Click(object sender, EventArgs e)  
{  
    conexion.Open();  
    string consulta = "delete from productos where codigo = '" + textBox1.Text + "'";  
    SqlCommand comando = new SqlCommand(consulta, conexion);
```

Ejecución

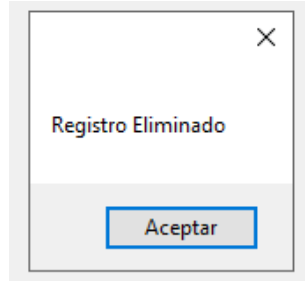
```
comando.ExecuteNonQuery();
```



Mensaje

El sistema elimina productos correctamente.

```
MessageBox.Show("Registro Eliminado");  
conexion.Close();
```



Funcionalidad 6: Evento btnModificar_Click

Consulta UPDATE

Permite modificar datos de productos existentes.

```
private void btnModificar_Click(object sender, EventArgs e)  
{  
    conexion.Open();  
    string consulta = "update productos set codigo='" + textBox1.Text + "', nombre='" + textBox2.Text + "', " +  
        "marca='" + textBox3.Text + "', precio='" + textBox4.Text + "' where codigo='" + textBox1.Text + "'";  
    SqlCommand comando = new SqlCommand(consulta, conexion);
```

Cantidad de registros afectados

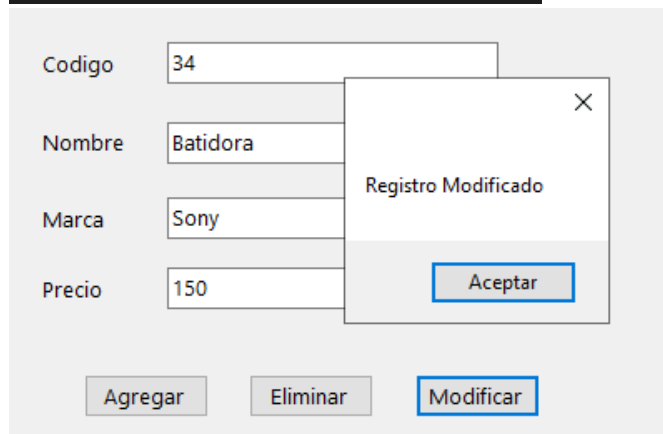
Guarda el número de registros modificados.

```
int cant;  
cant = comando.ExecuteNonQuery();
```

Validación de modificación

La modificación solo se confirma si realmente se actualizaron registros.

```
if (cant > 0)  
{  
    MessageBox.Show("Registro Modificado");  
}  
conexion.Close();
```





Programa: Negocio_version2:

1. Diseño del Programa

Mantenimiento de Productos

Codigo

Nombre

Marca

Precio

2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Mantenimiento de Productos
Label2	Text	Codigo
Label3	Text	Nombre
Label4	Text	Marca
Label5	Text	Precio
Button1	Name	btnAgregar
	Text	Agregar
Button2	Name	btnEliminar
	Text	Eliminar
Button3	Name	btnModificar
	Text	Modificar
dataGridView1	Name	dgvProductos

3. Código del Programa

Funcionalidad 1: Importación adicional

Permite trabajar con:

- DataTable
- ConnectionState

```
using Microsoft.Data.SqlClient;  
using System.Data;
```

Funcionalidad 2: Cargar productos automáticamente

Carga automáticamente los registros al iniciar la aplicación.

```
1 referencia  
private void Form1_Load(object sender, EventArgs e)  
{  
    CargarProductos();  
}
```

Funcionalidad 3: Conexión a la base de datos

Esta línea crea la conexión con SQL Server.

```
SqlConnection conexion = new SqlConnection(  
    "Server=localhost;Database=negocio;Integrated Security=true;Encrypt=false");
```

Elementos de la conexión

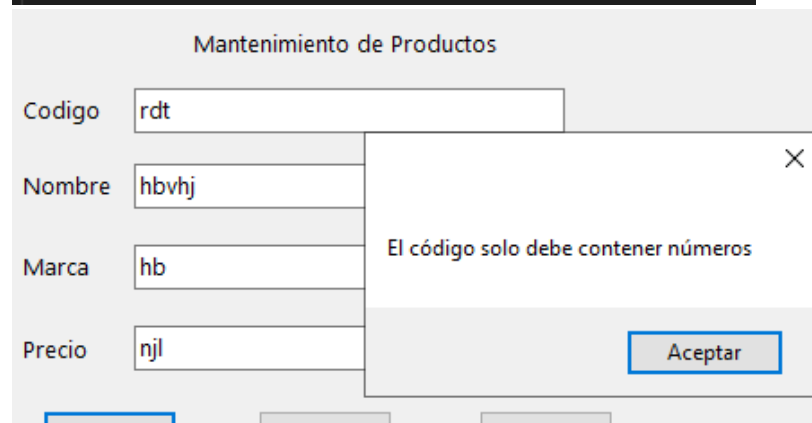
- Server=localhost: Servidor SQL local.
- Database=negocio: Base de datos utilizada.
- Integrated Security=true: Usa autenticación de Windows.
- Encrypt=false: Desactiva cifrado de conexión.

Funcionalidad 4: Evento btnAgregar_Click

Validación de código

Verifica que el código sea numérico.

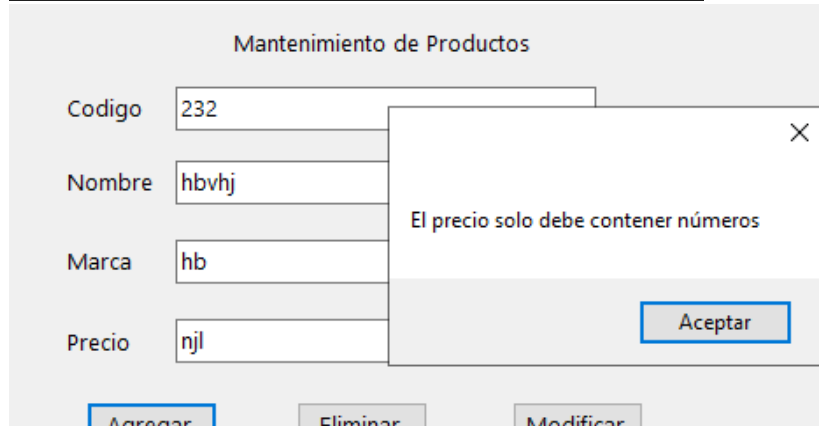
```
private void btnAgregar_Click(object sender, EventArgs e)  
{  
    // VALIDAR CODIGO  
    if (!int.TryParse(textBox1.Text, out _))  
    {  
        MessageBox.Show("El código solo debe contener números");  
        textBox1.Focus();  
        return;  
    }  
}
```



Validación de precio

Valida correctamente valores monetarios.

```
// VALIDAR PRECIO
if (!decimal.TryParse(textBox4.Text, out _))
{
    MessageBox.Show("El precio solo debe contener números");
    textBox4.Focus();
    return;
}
```



The screenshot shows a Windows application window titled "Mantenimiento de Productos". It contains four text boxes: "Codigo" with value "232", "Nombre" with value "hbjvhj", "Marca" with value "hb", and "Precio" with value "njl". Below these are three buttons: "Agregar", "Eliminar", and "Modificar". A modal message box is open over the "Precio" field, displaying the text "El precio solo debe contener números" and an "Aceptar" button.

Abrir conexión

```
try
{
    conexion.Open();
}
```

Consulta SQL INSERT

Inserta un nuevo producto en la tabla productos.

```
string consulta = "insert into productos values ('" + textBox1.Text + "','" + textBox2.Text + "','" + textBox3.Text + "','" + textBox4.Text + "')";
```

Creación del comando SQL

Asocia la consulta SQL con la conexión.

```
SqlCommand comando = new SqlCommand(consulta, conexion);
```

Ejecución de la consulta

Ejecuta consultas que no retornan datos.

```
comando.ExecuteNonQuery();
```

Mensaje de confirmación

```
MessageBox.Show("Registro Completado");
```

Mantenimiento de Productos

Codigo:

Nombre:

Marca:

Precio:

	CODIGO	NOMBRE	MARCA	PRECIO
	23	RFDES	GFD	343
	25	TV	Sony	2500
	34	Batidora	Sony	150
▶	43	Laptop	LG	2300

100 % No se encontraron problemas. Li

	CODIGO	NOMBRE	MARCA	PRECIO
1	23	RFDES	GFD	343
2	25	TV	Sony	2500
3	34	Batidora	Sony	150
4	43	Laptop	LG	2300

Registro Completado

Cerrar conexión

El sistema agrega correctamente nuevos productos en la base de datos y la tabla.

```
conexion.Close();
CargarProductos();
}
```

Manejo de excepciones

Evita que la aplicación se cierre inesperadamente.

```
catch (Exception ex)
{
    MessageBox.Show("Error: " + ex.Message);

    if (conexion.State == ConnectionState.Open)
        conexion.Close();
}
```

Funcionalidad 5: Evento btnEliminar_Click

Consulta DELETE

Elimina registros según el código ingresado.

```
private void btnEliminar_Click(object sender, EventArgs e)
{
    conexion.Open();
    string consulta = "delete from productos where codigo = '" + textBox1.Text + "'";
    SqlCommand comando = new SqlCommand(consulta, conexion);
}
```



Ejecución

```
comando.ExecuteNonQuery();
```

Mensaje

El sistema elimina productos correctamente.

```
MessageBox.Show("Registro Eliminado");  
conexion.Close();  
CargarProductos();  
}
```

Mantenimiento de Productos

Codigo: 23
Nombre: RFDES
Marca: GFD
Precio: 343

Agregar Eliminar Modificar Aceptar

	CODIGO	NOMBRE	MARCA	PRECIO
	23	RFDES	GFD	343
	25	TV	Sony	2500

Agregar Eliminar Modificar

	CODIGO	NOMBRE	MARCA	PRECIO
▶	25	TV	Sony	2500
	34	Batidora	Sony	150
*				

100 % No se encontraron problemas

Resultados Mensajes

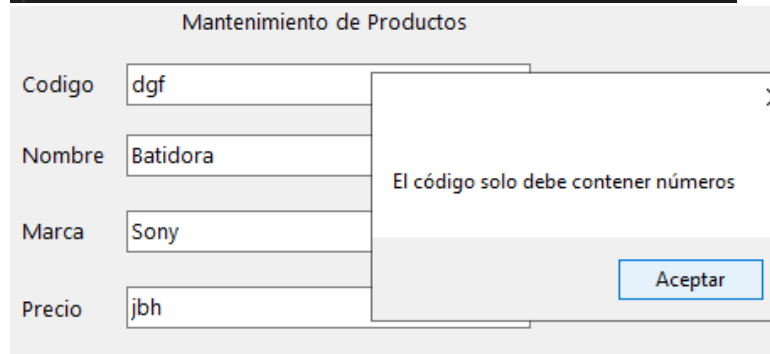
	CODIGO	NOMBRE	MARCA	PRECIO
1	25	TV	Sony	2500
2	34	Batidora	Sony	150

Funcionalidad 6: Evento btnModificar_Click

Validación de código

Verifica que el código sea numérico.

```
private void btnModificar_Click(object sender, EventArgs e)
{
    // VALIDAR CODIGO
    if (!int.TryParse(textBox1.Text, out _))
    {
        MessageBox.Show("El código solo debe contener números");
        textBox1.Focus();
        return;
    }
}
```



Mantenimiento de Productos

Codigo: dgh

Nombre: Batidora

Marca: Sony

Precio: jbh

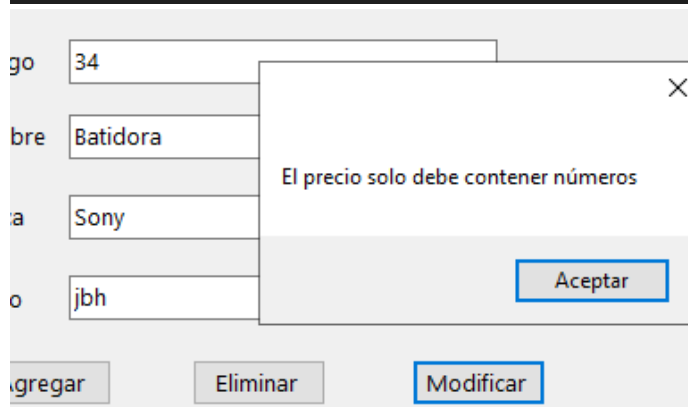
El código solo debe contener números

Aceptar

Validación de precio

Valida correctamente valores monetarios.

```
// VALIDAR PRECIO
if (!decimal.TryParse(textBox4.Text, out _))
{
    MessageBox.Show("El precio solo debe contener números");
    textBox4.Focus();
    return;
}
```



go: 34

bre: Batidora

a: Sony

o: jbh

El precio solo debe contener números

Aceptar

gregar Eliminar Modificar

Consulta UPDATE

Permite modificar datos de productos existentes.

```
try
{
    conexion.Open();
    string consulta = "update productos set codigo='" + textBox1.Text + "', nombre='" + textBox2.Text + "', " +
        "marca='" + textBox3.Text + "', precio='" + textBox4.Text + "' where codigo='" + textBox1.Text + "'";
    SqlCommand comando = new SqlCommand(consulta, conexion);
}
```



Cantidad de registros afectados

Guarda el número de registros modificados.

```
int cant;  
cant = comando.ExecuteNonQuery();
```

Validación de modificación

La modificación solo se confirma si realmente se actualizaron registros.

```
if (cant > 0)  
{  
    MessageBox.Show("Registro Modificado");  
}  
conexion.Close();
```

Funcionalidad 7: Método CargarProductos

Este método consulta y muestra todos los productos en el DataGridView.

```
private void CargarProductos()  
{  
    try  
    {  
        // Guarda el código del producto seleccionado (si hay uno)  
        string codigoSeleccionado = textBox1.Text;  
  
        using (SqlConnection conexion = new SqlConnection("server=localhost; database=negocio; integrated security=true; TrustServerCertificate=true"))  
        {  
            conexion.Open();
```

Consulta SELECT

Obtiene todos los registros almacenados.

```
string consulta = "SELECT * FROM productos";
```

SqlDataAdapter

Permite transferir datos desde SQL Server hacia la aplicación.

```
SqlCommand comando = new SqlCommand(consulta, conexion);  
SqlDataAdapter adaptador = new SqlDataAdapter(comando);
```

DataTable

Almacena temporalmente los datos obtenidos.

```
DataTable tabla = new DataTable();
```

Llenar y mostrar la tabla

Carga todos los registros dentro del DataTable y muestra los productos en el DataGridView.

```
adaptador.Fill(tabla);  
dgvProductos.DataSource = tabla;
```

Ajuste automático

Ajusta automáticamente las columnas al tamaño disponible.

```
// Configuración del autoajuste  
dgvProductos.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
```

Restaurar selección

Mantiene seleccionado el producto actual tras actualizar datos.

```
// Si había una selección, la restaura
if (!string.IsNullOrEmpty(codigoSeleccionado))
{
    foreach (DataGridViewRow row in dgvProductos.Rows)
    {
        if (row.Cells[0].Value?.ToString() == codigoSeleccionado)
        {
            dgvProductos.CurrentCell = row.Cells[0];
            break;
        }
    }
}
```

Funcionalidad 8: Evento dgvProductos_CellClick

Obtención de fila

```
private void dgvProductos_CellClick(object sender, DataGridViewCellEventArgs e)
{
    // Verifica que se haya hecho clic en una fila válida (no en el encabezado)
    if (e.RowIndex >= 0)
    {
        // Obtiene la fila seleccionada
        DataGridViewRow fila = dgvProductos.Rows[e.RowIndex];
    }
}
```

Carga en TextBox

Permite seleccionar registros y cargarlos automáticamente en los campos.

```
// Asigna los valores a los TextBox
textBox1.Text = fila.Cells["codigo"].Value?.ToString();
textBox2.Text = fila.Cells["nombre"].Value?.ToString();
textBox3.Text = fila.Cells["marca"].Value?.ToString();
textBox4.Text = fila.Cells["precio"].Value?.ToString();
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- Ambas aplicaciones hacen CRUD correctamente.
- La segunda versión es más segura y más estable.
- Las validaciones disminuyen los errores de ingreso.
- El DataGridView ayuda a visualizar la información.

¿Que se ha encontrado?

- El manejo de excepciones permite concretar la estabilidad del sistema.
- La conexión con SQL Server realiza un almacenamiento permanente de la información.
- Las validaciones habilitan evitar datos erróneos.
- La interfaz gráfica ayuda a interactuar al usuario.

CONCLUSIONES

- Se desarrolló correctamente un sistema CRUD conectado a SQL Server mediante C# y Windows Forms.
- La utilización de SqlConnection permitió obtener la conexión establecida con la base de datos.
- Con las instrucciones SQL se logró insertar, modificar y eliminar productos.
- La segunda versión mejoró notablemente la seguridad y estabilidad del sistema.
- El uso de las validaciones logró evitar los errores en el ingreso de datos numéricos.
- La utilización del DataGridView presenta registros de forma dinámicamente.
- El manejo de excepciones evita que ocurran fallos inesperados durante la ejecución.
- El presente proyecto permitió afianzar los conocimientos sobre bases de datos, SQL Server y la programación de aplicaciones de escritorio.

RECOMENDACIONES

- Implementar la búsqueda de productos.
- Agregar la confirmación antes de eliminar un registro.
- Implementar reportes de productos.



WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

rwestMSFT. (s.f.). SQL Server Management Studio (SSMS). Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/ssms/sql-server-management-studio-ssms>

Devart. (s.f.). What is ADO.NET - Architecture and Key Concepts. Devart Software. Recuperado de <https://www.devart.com/dotconnect/what-is-ado-net.html>